

AUDITORIA DE APLICAÇÃO WEB

Relatório de Auditoria de Segurança — Pratique + Pilates

Data: 14 de setembro de 2026

Projeto: pratique-pilates (kauanbg-dev/adm-pilates)

Escopo: código da branch de trabalho (site público, painel admin, API em Vercel Functions / Node, persistência Neon JSONB ou arquivos locais).

Stack detectada

Linguagem: JavaScript (ES modules) no Node.js.

Framework HTTP: handler único em `api/index.js` (Vercel Serverless) + `server.mjs` local.

ORM: nenhum. SQL via `@neondatabase/serverless` (tagged templates) ou JSON em `data/`.

Auth: cookie HttpOnly HMAC-SHA256 (`pratique_session`), senhas PBKDF2; gate extra

`ADMIN_ACCESS_KEY` só para o HTML do painel.

Frontend: HTML/CSS/JS estático (`index.html`, `admin.html`).

Deploy: `vercel.json`. Sem Docker, CI, Helm ou Terraform neste repositório.

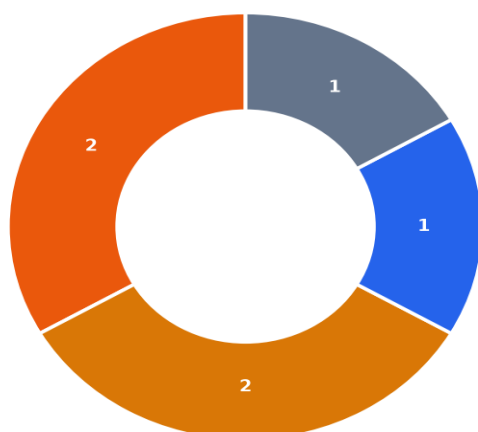
Nota metodológica. Cada categoria pedida foi mapeada assim: (1) Isolamento de inquilino → documento único `studio` + filtro por papel, não RLS/Supabase. (2) Permissão no navegador → cruzamento de `isStaff()` no `js/admin.js` com `mergeStudioWrite` / rotas em `api/index.js`. (3) IDOR → inventário de todos os handlers (`/api/login`, `logout`, `me`, `GET/PUT /api/studio`, `/api/painel`); não há rotas `/:id`. (4) Chaves → código, `.env.example`, `vercel.json`, `scripts`, histórico git e JS do frontend. (5) XSS → `innerHTML` no painel, site público e respostas HTML da API. Só entram achados com arquivo e linha no código atual. Sem Docker/CI/Helm, esses arquivos não foram forçados.

1. Resumo executivo

Foram verificados os cinco eixos no código real. Há 6 achados: nenhuma crítica, 2 altas (autorização de escrita do staff e XSS armazenado), 2 médias (segredos de exemplo e chave na URL), 1 baixa e 1 informativa. O modelo é de um único estúdio, não um SaaS multi-inquilino — IDOR clássico por ID de recurso não se aplica como rota HTTP.

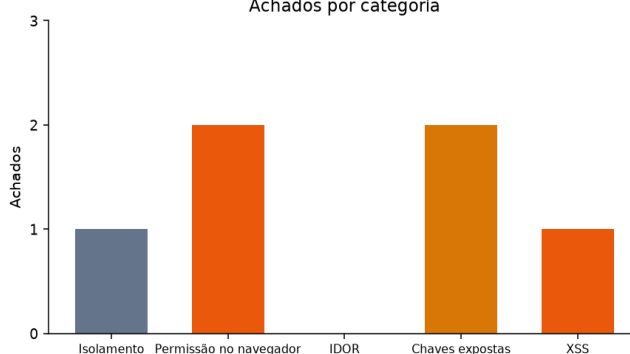
0 crítica	2 alta	2 média	1 baixa	1 informativa	6 total
--------------	-----------	------------	------------	------------------	--------------------

Achados por severidade



alta (2) baixa (1)
média (2) informativa (1)

Achados por categoria



2. Pontos fortes (o que está protegido)

- Sessão nas APIs de dados. GET/PUT /api/studio e /api/me exigem cookie HMAC válido (api/index.js:59-84). Login compara hash com timingSafeEqual (lib/auth.js:16-20). Cookie HttpOnly + SameSite=Lax.
- Financeiro do staff no servidor, não só na UI. studioForClient zera transactions no GET (lib/permissions.js:26-28). O PUT restaura as transações atuais (linhas 31-33). Cruzado com js/admin.js:337, 915-919, 1562+.
- Instrutores: dona imutável para staff. mergeStudioWrite reescreve a lista a partir do estado anterior e congela o perfil da Bia (lib/permissions.js:36-37). Staff não cria nem apaga instrutores pelo PUT.
- SQL parametrizado. Neon tagged templates em lib/store.js:76-105, 134, 144, 154 — sem concatenar e-mail/payload na query.
- Sem IDOR por rota /:id. Handlers existentes: login, logout, me, GET/PUT studio, painel HTML. Nenhum busca objeto por ID de path/query sem dono: o recurso é o documento único do estúdio.
- Segredos reais fora do git. .gitignore cobre .env e data/. Histórico sem .env commitado. JS do frontend sem API keys.
- ADMIN_PASSWORD obrigatório. lib/store.js:36-41 aborta se a env estiver vazia. Painel HTML 404 se ADMIN_ACCESS_KEY não existir (api/painel.js:15-36).
- Site público sem innerHTML de input. js/main.js usa textContent no formulário. API não monta e-mail HTML.
- Nomes de alunos escapados no painel. escapeHtml/escapeAttr em nome, e-mail, notas, descrição financeira (js/admin.js:1447-1456 e usos em 461, 499, 516, 958...).

- Path traversal local bloqueado para data/, .env, .git, lib/ (server.mjs:35-47). Rewrites Vercel equivalentes em vercel.json:10-11.

3. Pontos fracos (riscos centrais)

- A autorização fina (valor da aula, “só a Bia”) vive em grande parte no JavaScript do painel. O PUT copia o JSON do cliente.
- XSS armazenado: horários, status e IDs entram em innerHTML sem escape — um PUT autenticado executa script na sessão de quem abrir a agenda.
- A chave do painel viaja na query string; senhas de exemplo do repositório não são recusadas no boot.
- Não há multi-tenant: qualquer login válido vê (e, no caso staff, pode reescrever) o cadastro completo de alunos. Isso é o desenho do produto, mas o papel staff não é um isolamento de dados.

4. Achados detalhados por categoria

4.1 Isolamento de inquilino / dono

Mecanismo identificado: não há RLS (não é Supabase) nem tenant_id. Há uma linha studio.id = 1 (ou um studio.json). Isolamento = sessão autenticada + máscara por papel (transactions). Categoria multi-inquilino não se aplica como vazamento entre organizações.

4.2 Permissão definida no navegador

Cruzamento feito: financeiro, valor, novo instrutor, gerar cobranças, baixar/atrasar lançamento. Servidor cobre financeiro e lista de instrutores. Não cobre classPrice.

4.3 IDOR

Percorridos todos os handlers. Não existem rotas que leiam/alterem/deletem por ID de path, query ou body isolado. O PUT substitui o documento. Posse verificada só em transações (imutáveis para staff) e instrutores (merge). Nenhum achado clássico de IDOR. A reescrita ampla do JSON pelo staff está na categoria 2.

4.4 Chaves expostas

Sem Docker/CI/Helm/Terraform. Bundle frontend sem tokens. Histórico git: apenas placeholders em .env.example (commit 2f9d0a5).

4.5 Inputs sem tratamento (XSS)

Há helper de escape, mas a cobertura é irregular. Backend não interpola input em HTML de e-mail. Site público OK.

Sev.	Cat.	Arquivo:linha	Descrição
INFORMATIVA	Isolamento	lib/store.js:141-148; api/index.js:69-76; lib/permissions.js:26-28	GET /api/studio devolve o estúdio inteiro a qualquer login. Staff só perde transactions. Não há filtro por instrutor. Explorável como leitura de PII de todos os alunos por qualquer conta staff — alinhado ao produto de estúdio único, por isso informativo.
ALTA	Permissão	lib/permissions.js:31-48; api/index.js:79-93; js/admin.js:342, 1101-1102, 164-166	Staff não vê “Alterar valor”, mas o PUT grava classPrice e o restante do JSON (exceto transações/instrutores). Condição: sessão staff válida. PoC: PUT com classPrice: 0.

Sev.	Cat.	Arquivo:linha	Descrição
BAIXA	Permissão	lib/permissions.js:20-23	isOwnInstructor retorna true para id ins_alex sem checar o usuário. Hoje só há um staff; vira buraco se existir outro.
MÉDIA	Chaves	.env.example:1-6; lib/store.js:21-27 e 36-41	Placeholders públicos (escolha-uma-senha-forte, um-codigo-secreto-so-da-bia). Boot não rejeita esses valores. SESSION_SECRET pode faltar e cair em arquivo gerado.
MÉDIA	Chaves	api/painel.js:38-40; server.mjs:61	ADMIN_ACCESS_KEY na query ?k= (histórico, Referer, logs) e impressa no stdout do dev server. A API de login não usa essa chave — o risco combina com senha fraca.
ALTA	XSS	js/admin.js:266, 407, 466, 663, 713-714, 741, 751, 753, 758, 1293, 1612-1613, 1681	innerHTML com time/status/id/modalidade/data crus. Não há DOMPurify. Condição: atacante autenticado grava o payload; a vítima (ex.: admin) abre a agenda. Cookie HttpOnly não impede fetch com credentials.

4.6 Trechos citados

lib/permissions.js:31-33 — merge não filtra classPrice

```
export function mergeStudioWrite(incoming, current, user) {
  if (isAdmin(user) || !incoming || typeof incoming !== "object") return incoming;
  const next = { ...incoming, transactions: current.transactions || [] };
}
```

js/admin.js:266 — status sem escape

```
return `<span class="chip ${map[status]} || """>${labels[status]} || status</span>`;
```

api/painel.js:38-40 — chave na query

```
const offered = url.searchParams.get("k") || url.searchParams.get("acesso") || "";
const allowed = secretsEqual(offered, key) || readGate(token, sessionSecret(), key);
```

server.mjs:61 — chave no log

```
console.log(`Painel: http://127.0.0.1:${port}/admin.html?k=${process.env.ADMIN_ACCESS_KEY || "(veja data/access.key)"}`);
```

5. Recomendações priorizadas

- P1 — Autorização no PUT. Allowlist por papel. Recusar alteração de classPrice (e outros campos só-admin) para staff. Teste de regressão com conta Alex.
- P1 — XSS. Escapar todos os interpolados; validar time/status/id no servidor. Tratar innerHTML como lista de verificação, não exceção.
- P2 — Segredos. Recusar placeholders no boot; exigir SESSION_SECRET; tirar a chave da query e do log.
- P3 — isOwnInstructor. Ligar o perfil ao usuário autenticado, sem atalho ins_alex.
- P3 — Endurecimento. Rate limit no /api/login; Secure cookie sempre em produção; não devolver err.message cru no 500 (api/index.js:98 — observado, fora das cinco categorias).

6. Cobertura da auditoria (o que foi visto e está correto)

Handlers: POST /api/login, POST /api/logout, GET /api/me, GET /api/studio, PUT /api/studio, GET /api/painel (admin.html). Wrappers api/login.js, me.js, logout.js, studio.js só reexportam o handler. Frontend público (index.html + js/main.js) sem innerHTML de dados. Sem Dockerfile, .github/workflows, Helm ou Terraform — categoria de secrets nesses arquivos: não aplicável. Lib de sanitização: inexistente; o equivalente local é escapeHtml, aplicado em nomes/contatos e omitido em horários/status/IDs.

ISSUES PARA O GITHUB

Blocos prontos para colar. Achados triviais do mesmo tema (placeholders + chave na URL) foram agrupados na issue 3. O achado informativo de estúdio único não vira issue.

--- ISSUE 1 ---

[Segurança] Staff altera o valor da aula e o JSON do estúdio sem checagem no servidor

Labels sugeridas: `security`, `alta`

Problema

O painel esconde do instrutor (papel `staff`) as ações privilegiadas de **alterar o valor da aula**, ver financeiro, gerar cobranças e cadastrar instrutor. O servidor, no `PUT /api/studio`, só preserva `transactions` e restringe o array `instructors`. Qualquer outro campo do documento – inclusive `classPrice`, `clients`, `appointments`, `modalities` e `times` – é gravado como veio no body.

Por que é explorável

O staff já está autenticado. Basta um `PUT /api/studio` com o JSON completo (o próprio `Studio.save` envia o objeto inteiro). Não precisa da UI. Dá para zerar o valor da aula, apagar alunos ou reescrever a agenda.

Evidência

`js/admin.js:342` e `1101-1102` – a UI some para staff:

```
actions.innerHTML = isStaff() ? "" : "...editar-valor...";
function openClassPrice() {
  if (isStaff()) return;
  ...
```

`lib/permissions.js:31-48` – o merge de escrita não toca em `classPrice`:

```
export function mergeStudioWrite(incoming, current, user) {
  if (isAdmin(user) || !incoming || typeof incoming !== "object") return incoming;
  const next = { ...incoming, transactions: current.transactions || [] };
  // só instructors é filtrado
  ...
```

`api/index.js:79-93` – qualquer usuário logado pode PUT.

Impacto

Integridade financeira e operacional: preço cobrado, cadastro de alunos e grade de horários ficam sob controle de uma conta que a dona tratou como limitada.

Sugestão de correção

No servidor, para `staff`, recusar mudança de `classPrice` (e de qualquer campo que a UI já trate como só-admin). Ideal: allowlist de caminhos graváveis por papel, em vez de copiar o JSON inteiro.

Critérios de aceite

- [] `PUT /api/studio` com `classPrice` diferente, autenticado como staff, **não** altera o valor persistido
- [] A mesma chamada como admin continua funcionando
- [] Transações e o perfil da Bia continuam protegidos
- [] Teste automatizado ou script de verificação cobre o caso staff vs admin

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] XSS armazenado no painel via innerHTML sem escape

****Labels sugeridas:**** `security`, `alta`

Problema

O painel monta HTML com `innerHTML`. Existe `escapeHtml` / `escapeAttr`, mas vários campos que o usuário (ou um PUT autenticado) controla entram crus: `status` desconhecido, `time`, `id` em atributos, nome de modalidade no `<option>`, data e horários da grade.

Não há biblioteca de sanitização no projeto (`DOMPurify` etc.).

Por que é explorável

Um staff (ou admin comprometido) grava um agendamento com `time` ou `status` contendo HTML. Quando a Bia abre a agenda, o script roda ****no navegador dela****, com o cookie HttpOnly da sessão admin. `fetch('/api/studio', {credentials:'include'})` permite ler financeiro e gravar o estúdio como a dona.

Evidência

`js/admin.js:266` — status cai no HTML se não estiver no mapa:

```

`
return `<span class="chip ${map[status] || ""}>${labels[status] || status}</span>`;
`

```

Outros pontos: `407` (`\${a.time}`), `663`/`714` (`\${time}`), `713` `data-time="\${time}"`, `751` data no value, `753`/`758` options de horário/modalidade, `1293` horários da grade, `1612` modal de exclusão, `466`/`741` `data-id`/`value` de IDs.

Impacto

Roubo de sessão efetiva (ações no contexto da vítima), alteração do banco do estúdio, leitura de dados financeiros que o staff não deveria ver.

Sugestão de correção

Escapar ****todo**** interpolado em `innerHTML` (incluindo IDs, horários, status, datas). Validar no servidor formato de `time` (`HH:MM`), `status` (enum) e `id` (`[a-z0-9\_]+`). Preferir `textContent` / DOM APIs em vez de HTML concatenado.

Critérios de aceite

- [] Payload `**não** executa script
- [] `escapeHtml`/`escapeAttr` aplicados nos pontos listados, ou renderização sem `innerHTML`
- [] PUT rejeita campos fora do enum/formato

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Segredos de exemplo e chave de acesso na URL

****Labels sugeridas:**** `security`, `médica`

Problema

1. `.env.example` publica senhas e `ADMIN\_ACCESS\_KEY` previsíveis. O startup ****exige**** `ADMIN\_PASSWORD`, mas ****não rejeita**** o valor de exemplo. `SESSION\_SECRET` pode ser omitido e o código gera um arquivo local – na Vercel, se a env faltar, o comportamento é frágil.
 2. A chave do painel vai na query `?k=` / `?acesso=` (histórico, Referer, logs). O servidor de desenvolvimento ainda imprime a chave no stdout.

`.env` e `data/` não entram no git (ponto forte). Não há chave de API de terceiro no bundle do frontend.

Evidência

`.env.example:1-6` – `ADMIN\_PASSWORD=escolha-uma-senha-forte`, `ADMIN\_ACCESS\_KEY=um-codigo-secreto-so-da-bia`
 `lib/store.js:21-27` e `36-41` – secret opcional; senha só testa presença
 `api/painel.js:38` – `url.searchParams.get("k")`
 `server.mjs:61` – log com `ADMIN\_ACCESS\_KEY`

Impacto

Quem copia o exemplo para produção (ou deixa a chave na URL) facilita login e descoberta do HTML do painel. A API de login ****não**** depende da chave `k`; a senha fraca é o risco maior.

Sugestão de correção

- Recusar no boot senhas/chaves iguais às do `.env.example`
- Exigir `SESSION\_SECRET` forte também na Vercel
- Trocar o gate `?k=` por POST / header / cookie sem query string
- Não logar a chave

Critérios de aceite

- [] App recusa subir com `ADMIN\_PASSWORD`/`ADMIN\_ACCESS\_KEY`/`SESSION\_SECRET` iguais ao exemplo
- [] Painel não exige a chave na URL
- [] `server.mjs` não imprime o valor de `ADMIN\_ACCESS\_KEY`
- [] Histórico git continua sem `.env` real

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Perfil ins_alex editável por qualquer usuário staff

****Labels sugeridas:**** `security`, `baixa`

Problema

`isOwnInstructor` devolve verdadeiro se `instructor.id === "ins\_alex"`, ****sem**** comparar com o usuário autenticado. Hoje só existe um staff (Alex), então o efeito prático é pequeno. Se no futuro houver outro instrutor com papel `staff`, ele poderá alterar o cadastro do Alex.

Evidência

`lib/permissions.js:20-23`:

```

```
export function isOwnInstructor(instructor, user) {
 if (!instructor || !user) return false;
 if (String(instructor.id || "") === "ins_alex") return true;
 return String(instructor.name || "").trim().toLowerCase() === String(user.name || "").trim().toLowerCase();
}
```

**## Impacto**

Baixo no estado atual (um único staff). Vira falha de autorização se o time crescer.

**## Sugestão de correção**

Remover o atalho `ins\_alex`. Autorizar só por `user.email` / id estável ligado ao instrutor, não por nome.

**## Critérios de aceite**

- [ ] Staff cuja conta não é a do Alex **\*\*não\*\*** altera o registro `ins\_alex` no PUT
- [ ] Alex continua podendo editar o próprio perfil
- [ ] Perfil da Bia (`ins\_bia`) permanece imutável para staff

**--- FIM ISSUE 4 ---**